

MINI-JIRA CLI TASK MANAGER

Rappresentazione del progetto pratico #1

Applicazione CLI in Python per la gestione del lavoro di un team di sviluppo

Progetto: **progetto_pratico_1/**

Obiettivo: trasformare regole tipiche della gestione di un team software in comportamenti verificabili direttamente dal programma.

1. Il problema che risolve

Mini-Jira simula la gestione operativa di un piccolo team di sviluppo. L'applicazione permette di gestire **utenti, ruoli, ticket, assegnazioni, Sprint, commenti, storico, carico di lavoro e statistiche**.

Il punto centrale non è riprodurre graficamente Jira, ma applicare in modo automatico le **regole di business** che governano il lavoro del team.

2. Ruoli e responsabilità

Ruolo	Responsabilità principali
Project Manager (PM)	Crea ticket, assegna attività, crea e gestisce gli Sprint, gestisce il team e il limite di carico.
Developer (DEV)	Visualizza e commenta i ticket assegnati e avanza lo stato dei propri ticket.

3. Ciclo di vita dei ticket

Il workflow è controllato e non permette di saltare gli stati:

TODO → IN_PROGRESS → IN_REVIEW → DONE

- Ogni transizione deve rispettare l'ordine previsto.
- Solo il **Developer assegnatario** può avanzare il ticket.
- Un ticket non può avanzare se non è assegnato.
- Quando arriva a **DONE**, il carico del Developer viene decrementato automaticamente.

4. Gestione del carico di lavoro

Il limite di ticket per Developer è centralizzato in **ConfigurazioneTeam**. Il valore predefinito è **3 ticket**.

```
Developer: pippo
Carico: 3 / 3
Nuova assegnazione → ERRORE
LimiteCaricoLavoroSuperatoError
```

Se il limite è raggiunto, il sistema impedisce nuove assegnazioni. Il carico si libera automaticamente quando un ticket arriva a DONE.

5. Tipi di ticket e OOP

Il modello utilizza una classe base **Ticket** e due specializzazioni:

Classe	Caratteristica	Complessità
BugTicket	Severità: BLOCKS, CRITICAL, MINOR	URGENTE oppure Bugfix Standard
FeatureTicket	Story Points	Alta Complessità da 8 SP; Media/Bassa sotto

Il metodo **calcola_complessita()** viene implementato in modo diverso nelle classi derivate: è un esempio concreto di **ereditarietà e polimorfismo**.

6. Sprint, commenti e storico

Gli Sprint raggruppano ticket con un obiettivo comune e hanno un proprio ciclo di vita: creazione, avvio e chiusura. Il sistema impedisce di inserire lo stesso ticket in più Sprint contemporaneamente.

Ogni ticket conserva **commenti e storico delle operazioni**, così è possibile ricostruire le modifiche principali effettuate durante il lavoro.

7. Architettura del progetto

```

progetto_pratico_1/
  ■■■ main.py
  ■■■ modelli/
    ■ ■■■ utente.py
    ■ ■■■ ticket.py
    ■ ■■■ sprint.py
    ■ ■■■ configurazione_team.py
  ■■■ servizi/
    ■ ■■■ gestionale_jira.py
    ■ ■■■ report_service.py
    ■ ■■■ logger.py
  ■■■ utilita/validazioni.py
  ■■■ eccezioni/eccezioni_custom.py
  ■■■ tests/
    ■■■ test_utente.py
    ■■■ test_ticket.py
    ■■■ test_sprint.py
    ■■■ test_configurazione_team.py
    ■■■ test_gestionale.py
    ■■■ esegui_test.py

```

La struttura separa **modelli, servizi, validazione, eccezioni, interfaccia e test**. Il GestionaleJira coordina le operazioni che coinvolgono più entità.

8. Autenticazione e permessi

Permesso	DEV	PM
Visualizzare ticket	✓	✓
Commentare ticket	✓	✓
Cambiare stato	✓	—
Creare ticket	—	✓
Assegnare ticket	—	✓

Creare / gestire Sprint	—	✓
Gestire il team	—	✓

Il PM non cambia lo stato del ticket. L'avanzamento è responsabilità del Developer assegnatario, verificata dal gestionale.

9. Dati, logging e gestione degli errori

L'applicazione utilizza un'architettura **in-memory**: utenti, ticket, Sprint e configurazione rimangono in RAM durante l'esecuzione. Alla chiusura del programma lo stato non viene mantenuto.

L'unico dato persistente è il log delle operazioni, scritto in **logs/app_jira.log** tramite FileLogger.

Gli errori di dominio utilizzano una gerarchia di eccezioni basata su **JiraException**, con casi specifici per permessi, duplicati, elementi inesistenti, limite di carico e transizioni non valide.

10. Testing

La suite di test è organizzata per modulo e utilizza funzioni di test con **assert**, senza dipendere da unittest.

Sono verificati sia i casi normali sia i casi di errore: dati non validi, duplicati, permessi insufficienti, assegnazioni non consentite, superamento del limite di carico, transizioni illegali e operazioni non valide sugli Sprint.

Il README fornito non indica il numero totale dei test superati; per questo il PDF non inventa una quantità di test.

11. Demo consigliata

```

PM → crea ticket → assegna al DEV
      ↓
DEV → TODO → IN_PROGRESS → IN_REVIEW → DONE
      ↓
    carico decrementato
      ↓
    storico + log aggiornati

```

12. Conclusione

Mini-Jira CLI Task Manager dimostra come progettare un piccolo gestionale modulare partendo da requisiti concreti e trasformando le regole del dominio in controlli software.

Punti chiave: separazione delle responsabilità, OOP, gestione dei permessi, workflow controllato, limite del carico, Sprint, logging, eccezioni personalizzate e testing.

Tecnologia: **Python 3.10+** · Nessuna dipendenza esterna · Interfaccia CLI